

Ali Alemami

Amman, Jordan | ali.alemami01@gmail.com | +962 780 020 203 | github.com/ali-alemami | ali-alemami.github.io | linkedin.com/in/ali-alemami

Computer Science student specializing in backend & low-level systems engineering, with hands-on experience in **C/C++** (POSIX system APIs, multithreading, concurrency, memory models) and **C#.NET** (3-tier architecture, ADO.NET, SQL Server). Focused on backend architecture, POSIX systems programming, and high-reliability software engineering.

EDUCATION

42 Amman — Common Core, 22/24 projects complete (*peer-reviewed, project-based*)

2025 – Present

The Hashemite University — BSc in Computer Science

2022 – 2026 (Expected)

TECHNICAL SKILLS

Programming Languages:	C (POSIX/C99), C++ (98/11/17), C# (.NET), SQL (T-SQL), Bash
Systems & OS Concepts:	POSIX APIs, Multithreading (pthreads), Concurrency & Mutexes, Sockets / epoll, Process Control & IPC, Memory (Valgrind/ASan)
CS Fundamentals:	Data Structures & Algorithms, OOP & Design Patterns, 3-Tier Architecture, Computer Networks
Tools & Infrastructure:	SQL Server, ADO.NET, Docker, Docker Compose, NGINX, Git, Linux, GDB, Makefiles
Languages (Spoken):	Arabic (Native), English (Fluent / Professional)

42 CORE CURRICULUM — 22/24 PROJECTS COMPLETE

- **Multithreading & Concurrency** (C — [philo](#)): Engineered a race-free simulation of Dijkstra's Dining Philosophers using POSIX threads (pthreads) and mutex locks; built dedicated asynchronous monitoring routines and microsecond-precision delta timers (gettimeofday) preventing thread starvation and deadlocks.
- **HTTP Web Server** (C++98 — [webserv](#)): Engineering a non-blocking HTTP/1.1 server using epoll I/O event multiplexing; implementing stateful request stream parsing (GET/POST/DELETE), NGINX-style configuration parsing, client timeout handling, and CGI execution pipelines.
- **DevOps & Container Infrastructure** ([inception](#)): Architected a multi-container infrastructure with Docker Compose on Alpine Linux; configured isolated containers for NGINX with TLS/SSL encryption (v1.2/v1.3), WordPress (PHP-FPM), and MariaDB with dedicated bind mounts, internal bridge networking, and PID 1 entrypoint signal management.
- **Unix Shell & AST Engine** (C — [minishell](#)): Implemented a POSIX-compliant shell with an Abstract Syntax Tree (AST) parser to enforce operator precedence (|, &&, ||), nested subshell forks, quote-aware tokenization, heredocs with delimiter expansion, variable expansion, and signal handling; zero leaks via Valgrind.
- **3D Graphics & Raytracing Engine** (C — [mini_rt](#)): Built a 3D raytracing renderer from scratch in C; implemented vector math (dot/cross products, normalization), quadratic discriminant ray-surface intersections (spheres, planes, cylinders), and Phong reflection models (ambient, diffuse, specular highlights, and hard shadows) rendered via MiniLibX.
- **C++ Modules & Ford-Johnson Algorithm** (C++ — [cpps](#)): Completed 42 C++ Modules 00–09 (OOP, Orthodox Canonical Form, polymorphism, templates, STL); implemented the **Ford-Johnson Merge-Insert Sort (CPP09)** optimizing comparison steps using dual STL sequence containers (std::vector, std::deque) and Jacobsthal sequence pairing heuristics.
- **Foundations & Algorithms**: [push_swap](#) (bitwise radix sort across two stacks), pipex (multi-pipe fork/dup2), fract-ol (Mandelbrot/Julia sets), minitalk (UNIX signal bit-shift IPC), libft, ft_printf, get_next_line. In progress: transcendence (real-time multiplayer web app).

SOFTWARE DEVELOPMENT & DATABASE PROJECTS

Programming Advices roadmap (Dr. Mohammed Abu-Hadhoud) — 17/24 courses completed in C#.NET, SQL Server, and C++

- **Bank & ATM CLI System** (C++ — [Bank-and-ATM-System-CPP](#)): Built a terminal-based banking and ATM system in C++ with bitmask user permissions (bitwise flags), transaction audit logging, and file-based data persistence.
- **Contacts Management App** (C#, ADO.NET, SQL Server — [Contacts-App-3Tier](#)): Developed a 3-tier desktop application separating UI, business logic, and data access layers; utilized parameterized T-SQL queries to ensure data integrity and prevent SQL injection.
- **Relational Database Schemas & SQL** (SQL Server — [Programming-Advices-Core](#)): Designed 5 relational database schemas (Medical Clinic Booking, E-Commerce Store, HR/Payroll, Vehicle Registry, Employee Analytics) with primary/foreign key constraints, normalization, and multi-table analytical JOIN queries.
- **Algorithms & Problem Solving** (C++ — 225+ Problems): Implemented solutions across 5 progressive difficulty levels for data structures, matrix operations, string parsing, combinatorial problems, and date/time arithmetic algorithms.
- **Windows Forms Projects** (C#, .NET — [CSharp-WinForms-Collection](#)): Built 10+ desktop utility and game applications in C# focusing on UI event handling, input validation, and object-oriented design principles.